

Data Session 9, part 2: What Is a 70% Chance?

84-355 Democracy's Data | Forecasting models and prediction markets

Jonathan Cervas

August 11, 2026

Everything here runs as given. Your job is to read what comes out and say what it means.

Where did that number come from?

“Candidate X has a 72% chance of winning.”

This morning you took a poll apart. A poll reports a **share** — 48% of respondents said this. Nobody votes on a share. Between the poll and the sentence above sits a **model**, and next to the model sits a **market**, and the two produce the same kind of object: a probability.

Two weeks from today the country votes, and every one of these numbers will be graded. So: **what would it even mean for “72%” to be right?**

If the candidate loses, the forecast was not refuted. 72% is not 100%. Things priced at 72% are supposed to fail 28% of the time.

A single probability cannot be checked. Many of them can — and that is what we do today.

Part 1 — The problem with checking one

```
p <- 0.72
c(if_it_happens = 1 - (1 - p)^2,
  if_it_doesnt  = 1 - p^2)
```

```
#> if_it_happens  if_it_doesnt
#>           0.9216           0.4816
```

Whatever happens, a defender of the forecast has a reply. **This is not a weakness of forecasters — it is a property of probability**, and it is why “the polls were wrong in 2016” is a much emptier sentence than it sounds.

The only escape is volume. Collect hundreds of forecasts, group them by what they claimed, and check the groups.

Part 2 — 746 forecasts with known answers

A prediction market is a forecast with money behind it: people buy contracts that pay \$1 if a thing happens, so the price is what the crowd thinks the chance is. When it resolves, we find out.

```
d <- read.csv("data/polymarket_resolved.csv", stringsAsFactors = FALSE)

c(markets = nrow(d),
  resolved_yes = sum(d$outcome),
  priced_under_10pct = sum(d$p7 < 0.10))
```

```
#>           markets      resolved_yes priced_under_10pct
#>           746           145           527
```

746 resolved markets, each with the price **seven days before it closed** and what actually happened.

Note the third number. **70.6% of these markets were priced under 10%** — most questions people trade are about things that do not happen. Keep that in mind when you read the table below.

```
head(d[order(-d$volume), c("question", "p7", "outcome")], 8)
```

```
#>                                     question      p7
#> 1      Will Donald Trump win the 2024 US Presidential Election? 0.6585
#> 2      Will Kamala Harris win the 2024 US Presidential Election? 0.3485
#> 3                                     Will Donald Trump be inaugurated? 0.9935
#> 4      Will the Sacramento Kings win the 2025 NBA Finals? 0.0005
#> 5      MicroStrategy sells any Bitcoin by May 31, 2026? 0.0015
#> 6 Will Nicolae Ciucă win the 2024 Romanian Presidential election? 0.0015
#> 7                                     US forces enter Iran by April 30? 0.9995
#> 8      Will Zelensky wear a suit before July? 0.0365
#>      outcome
#> 1          1
#> 2          0
#> 3          1
#> 4          0
#> 5          0
#> 6          0
#> 7          1
#> 8          0
```

The 2024 presidential market is the first row: seven days out it gave Trump **0.6585**, and the outcome was 1.

Part 3 — Calibration

Group the markets by the price they carried, then ask what share actually happened. **If the prices mean anything, the two columns should match.**

```

b <- cut(d$p7, breaks = seq(0, 1, 0.1), include.lowest = TRUE)
tab <- data.frame(
  band      = levels(b),
  n         = as.vector(table(b)),
  priced    = round(as.vector(tapply(d$p7, b, mean)), 3),
  happened   = round(as.vector(tapply(d$outcome, b, mean)), 3))
tab$gap <- round(tab$happened - tab$priced, 3)
tab

```

```

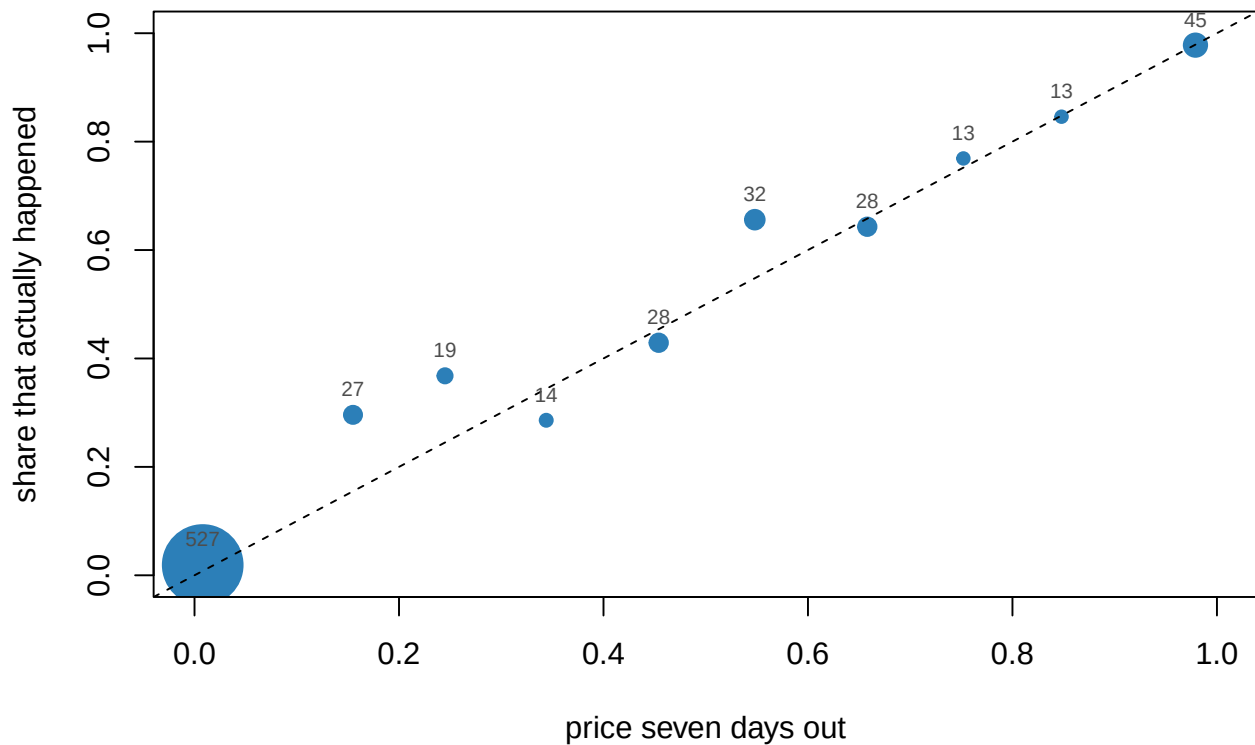
#>      band      n priced happened  gap
#> 1  [0,0.1] 527  0.008   0.019 0.011
#> 2 (0.1,0.2]  27  0.155   0.296 0.141
#> 3 (0.2,0.3]  19  0.245   0.368 0.123
#> 4 (0.3,0.4]  14  0.344   0.286 -0.058
#> 5 (0.4,0.5]  28  0.454   0.429 -0.025
#> 6 (0.5,0.6]  32  0.548   0.656  0.108
#> 7 (0.6,0.7]  28  0.658   0.643 -0.015
#> 8 (0.7,0.8]  13  0.752   0.769  0.017
#> 9 (0.8,0.9]  13  0.848   0.846 -0.002
#> 10 (0.9,1]  45  0.979   0.978 -0.001

```

```

plot(tab$priced, tab$happened, pch = 19, cex = sqrt(tab$n) / 4,
     xlim = c(0, 1), ylim = c(0, 1), col = "#2c7fb8",
     xlab = "price seven days out", ylab = "share that actually happened")
abline(0, 1, lty = 2)
text(tab$priced, tab$happened, tab$n, pos = 3, cex = 0.7, col = "grey30")

```



Read the top of the table first. Markets priced 0.8–0.9 happened **84.6%** of the time. Priced 0.9–1.0, **97.8%**. Priced 0.7–0.8, **76.9%**. Those are close enough to the line that the prices are doing real work.

```
c(brier_market = round(mean((d$p7 - d$outcome)^2), 4),
  brier_always50 = round(mean((0.5 - d$outcome)^2), 4),
  brier_baserate = round(mean((mean(d$outcome) - d$outcome)^2), 4))
```

```
#>   brier_market brier_always50 brier_baserate
#>         0.0644         0.2500         0.1566
```

The **Brier score** is the average squared error of a probability — lower is better. The market scores **0.0644**, against **0.25** for saying “50%” every time and **0.1566** for always guessing the overall base rate. **The market is carrying a great deal of information.**

Part 4 — Where it looks wrong, and whether to believe that

Now the bottom of the table. Markets priced 0.1–0.2 happened **29.6%** of the time — roughly twice what they claimed. That looks like a real bias.

Before believing it, count how many chances we gave ourselves to find one.

```
for (lv in levels(b)) {
  k <- b == lv; n <- sum(k)
  if (n < 5) next
  p <- binom.test(sum(d$outcome[k]), n, mean(d$p7[k]))$p.value
  cat(sprintf(" %-10s n=%3d expected %5.1f observed %3d p=%.3f%s\n",
              lv, n, sum(d$p7[k]), sum(d$outcome[k]), p,
              ifelse(p < 0.05, " *", "")))
}
```

```
#>   [0,0.1]   n=527 expected   4.4 observed  10   p=0.015  *
#>   (0.1,0.2] n= 27 expected   4.2 observed   8   p=0.058
#>   (0.2,0.3] n= 19 expected   4.7 observed   7   p=0.282
#>   (0.3,0.4] n= 14 expected   4.8 observed   4   p=0.783
#>   (0.4,0.5] n= 28 expected  12.7 observed  12   p=0.851
#>   (0.5,0.6] n= 32 expected  17.5 observed  21   p=0.287
#>   (0.6,0.7] n= 28 expected  18.4 observed  18   p=0.844
#>   (0.7,0.8] n= 13 expected   9.8 observed  10   p=1.000
#>   (0.8,0.9] n= 13 expected  11.0 observed  11   p=1.000
#>   (0.9,1]   n= 45 expected  44.1 observed  44   p=0.615
```

We tested **10 bands**. At a 5% threshold, you expect roughly one false positive from pure chance even if the market is perfectly calibrated.

Exactly one band clears it: the longshots, [0, 0.1] — and it is not a small band. It holds 527 of the 746 markets ($p = 0.015$). The next band up, (0.1, 0.2], has 27 markets and $p = 0.058$; the one after that has 19 and $p = 0.282$, which is no evidence of anything.

So the honest reading is **not** “the market misprices longshots across the board.” It is that the one place a real deviation shows up is the place where almost all the contracts are, and even there the effect is small in absolute terms: priced 0.008, happened 0.019.

```
k <- d$p7 > 0.1 & d$p7 <= 0.3
binom.test(sum(d$outcome[k]), sum(k), mean(d$p7[k]))$p.value
```

```
#> [1] 0.03674446
```

Pooling the 0.1–0.3 range gives $p = 0.037$ — significant by convention, and not by much, on 46 markets.

The honest summary is: the market is well calibrated where most of the money is, and there is a hint that low-probability events happen more often than priced, which this sample is too small to establish. A forecaster who claimed either “markets are perfectly calibrated” or “markets systematically underprice longshots” would be going beyond this evidence.

Part 5 — Your turn

Change **one** thing, re-run, and write about what changed. Pick one.

Option A — A different moment. The table uses the price seven days out. Redo it with `p30` — thirty days out — and drop rows where `p30` is missing. Is the market worse further from resolution? It should be. Is it?

Option B — Only the big markets. Restrict to markets with volume above the median and redo the calibration. More money should mean better prices. Does it?

Option C — Score yourself. The Brier score works on any forecaster. Pick five of the questions in `d` you could plausibly have answered a week out, write down your own probabilities *before* looking at the outcome column, and compute your Brier score against the market’s. Report honestly.

Option D — Your own question. 746 markets, two snapshots, one outcome column.

What this lab cannot tell you

- **Whether models beat markets.** We only have the market. **Forecasting model outputs are not published as downloadable data** — no file exists to check 538, Silver Bulletin or Split Ticket against this the way we just checked Polymarket. That absence is the same shape as the ad libraries in Session 5: the claim is public, the evidence behind it is not.
- **Anything specific to politics.** These are the highest-volume resolved markets, and they include sport, crypto and Fed decisions. What is being tested is the **market mechanism**, not American elections.
- **Whether the price was right *earlier*.** Seven days out is one arbitrary moment. A market that was badly wrong for a year and correct at the end scores well here.
- **Whether anyone can beat it.** Calibration says the prices are honest, not that they are unbeatable — and a market that is well calibrated overall can still be badly wrong about any single question, including the one you care about.

On the discussion board

By **Friday, 11:59 p.m.** A sentence or two is enough.

Three that would make good posts:

- Things priced at 90% happened 97.8% of the time and things priced at 15% happened 29.6% of the time. Which of those two numbers should worry you more?
- If a single forecast cannot be wrong, what is a forecast *for*?
- The country votes in two weeks. What number would you want to see published alongside every forecast, and why is nobody publishing it?

Where the data came from

Polymarket, via two public APIs that need no key: the Gamma markets API for resolved markets and outcomes, and the CLOB price-history API for the price before resolution. 746 markets survived the filters, out of 900 pulled. The build is in `data/build-data.py`.

Markets are included only if they settled cleanly to Yes or No — unresolved, void and scalar markets cannot be scored and were dropped.